WHY EMBEDDED CHECKS DON'T SCALE

The Customer Collaboration Platform relied on code-level checks to enforce boundaries. Each request handler verified that a principal either belonged to ACME or was associated with a specific, current project. The system used project IDs as a convenient shortcut, an integer that linked employees, customer accounts, and documents. As long as ACME ran the platform internally, that system worked: one namespace, one database, one source of truth.

But a project ID is not the same as a tenant boundary. It is simply a shared field that is trusted implicitly across every part of the application. There was no central authority verifying which IDs existed, no separate policy space for each project, and no straightforward way to isolate data or identities. The code assumed that *project equality* meant authorization, a fragile assumption that held only because everyone operated within ACME's trust boundary.

This tight coupling between application logic and access rules made the system brittle. Adding a new rule required modifying the application code. Supporting federation or new customer types involved writing exceptions. Because the logic was spread across dozens of microservices, there was no central point to observe or analyze access decisions. Auditing access was therefore impossible.

Externalizing authorization decisions makes the process dynamic. Rather than hardcode who can do what, the application delegates those decisions to a policy engine that evaluates policies based on current data, identity, and context at runtime. This method converts *ad hoc* checks into structured rules, provides true tenant isolation, and enables governance to evolve without altering application code.

13.3 Identity and tenancy foundation for C³

When ACME began designing the Customer Collaboration Cloud (C³), the first challenge wasn't creating new policies. Instead, it was building the